Learning Objectives

Learners will be able to...

- **Define a Data Structure**
- **Differentiate Linear and Non-Linear Data Structures**
- **Perform Basic Operations(insertion,deletion,merging) with Arrays**

Data Structures

What is a Data Structure?

Let's break down the two fundamental words in **Data Structures**: `Data` and `Structure`.

Data: In the simplest terms, data is information. In the context of computer science, `data` refers to the quantities, characters, or symbols on which operations are performed by a computer. `Data` can exist in various forms - it could be numeric values, text, images, audio, video, and even complex structured types like lists, arrays, and so forth.

Structure: Structure implies the arrangement or organization of components. In the world of computer science, when we refer to the `structure`, we're talking about how we are organizing or arranging our data in the computer's memory for efficient access and modification.

definition

Data Structures are particular ways of organizing data in a computer so that we can perform different operations. Some of those basic operations include:

- Insertion
- Deletion
- Merging
- Searching
- Sorting
- Traversal

We can categorize data types into two categories:

- **Primitive data types**
- **Non-primitive data types**

The primitive type of data types includes the predefined data structures such as `char`, `float`, `int`, and `double`. Non-Primitive data types are composed of primitive data types.

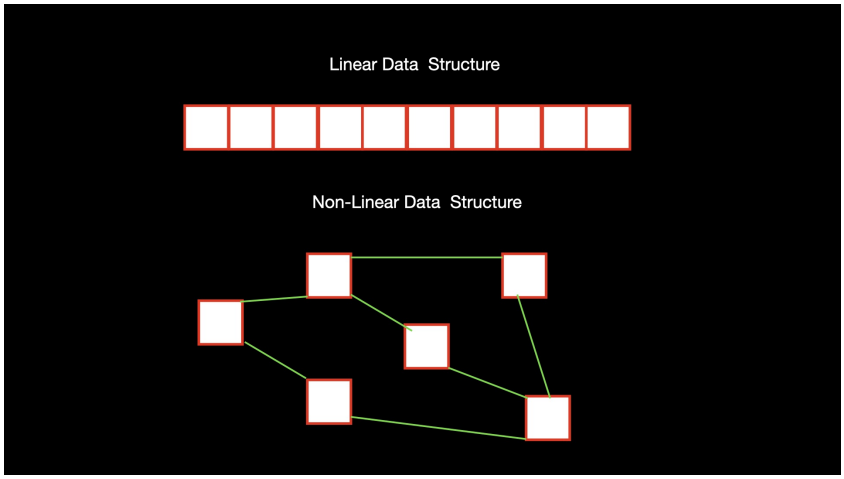
Simple data structures are types of data structures that are generally built from primitive data types like `int`, `float`, `double`, `string`, `char`.

Compound data structures are types that the user can build by combining simple data structures.

This data structure can be further categorized into **Linear and Non-Linear data structures**.

Linear data structure – It is a type of data structure where data is stored and managed in a linear sequence. Some of the popular linear data structures that we widely use in C++ are arrays, stacks, queues, and linked lists

Non-Linear data structures – The data structures of this type are basically multilevel data structures. The data are arranged in a manner that does not follow a linear fashion. Some of the popular non-linear data structures that we widely use are trees and graphs.



Algorithms

Algorithms

Now that we are more or less familiar with the Data structures bit of “data structures and algorithms,” we can tackle the question, “What is an algorithm?”

definition

An **algorithm** is a step-by-step procedure or a set of rules to be followed in calculations or other problem-solving operations, especially by a computer. In simpler terms, it’s a recipe for accomplishing a task.

For example, an algorithm could be a process for sorting a list of names alphabetically, calculating the square root of a number, finding the shortest path in a graph, or any other sequence of operations that can be programmed and implemented.

The study of algorithms and data structures go hand in hand. Data structures are often designed to facilitate specific algorithms, and algorithms are often created to process specific data structures. Together, they form the foundation of computer programming and are essential for the design of efficient software.

Let’s try an example with arrays that we should be familiar with. An array is a data structure, therefore we are looking to be able to perform the following operations to it: traversal, insertion, deletion, merging.

We will start with a basic array traversal. The goal is to examine each element of a given array, one by one, until we reach the last element. To achieve this, we will use a loop to iterate through the array and access each element sequentially.

```
int main() {
    int array[] = {1, 2, 3, 4, 5};

    // Iterate through the array using a for loop.
    for (int i = 0; i < sizeof(array)/sizeof(array[0]); i++) {
        std::cout << array[i] << std::endl;
    }

    return 0;
}
```

In this course, “**Data Structures and Algorithms**,” we will explore the symbiotic relationship between these two essential aspects of computer science. We’ll dive deeper into various types of data structures, understand their internal workings, learn when to use which data structure, and examine their space and time complexities. For the rest of this lesson, we will tackle different operations with the array data structure.

Insertion

Insertion in Arrays

When it comes to arrays, **insertion** refers to the process of adding a new element to the array at a specific position. However, since arrays in C++ can have a fixed size, we don't have a direct method to add an element to it.

To perform an insertion operation, we usually have to create a new array with a size larger than the original array by one and then copy elements from the old array to the new one. After copying, we can add the new element at the desired location.

Here's an example of how you could do it:

```cpp
int main() {

    int array[] = {1, 2, 3, 4, 5};
    int insertValue = 10;
    int insertIndex = 2;

    // print original array
    std::cout << "Original Array:\n";
    for (int i = 0; i < sizeof(array)/sizeof(array[0]); i++) {
        std::cout << array[i] << " ";
    }
    std::cout << "\n";

    // create a new array of size array.length + 1
    int newArraySize = sizeof(array)/sizeof(array[0]) + 1;
    int newArray[newArraySize];

    // copy elements from old array to new array, leaving space
        for new element
    for (int i = 0; i < insertIndex; i++)
        newArray[i] = array[i];

    // add new element
    newArray[insertIndex] = insertValue;

    // copy the rest of elements from old array to new array
    for (int i = insertIndex + 1; i < newArraySize; i++)
        newArray[i] = array[i - 1];

    // print new array
    std::cout << "Array after insertion:\n";
    for (int i = 0; i < newArraySize; i++) {
        std::cout << newArray[i] << " ";
    }
    return 0;
}
```

However, this method is not efficient, especially for large arrays or for multiple insertions. In C++, we would typically use a dynamic data structure, such as a vector, for these types of operations, which we will explore later in the course.

Deletion

Deletion in Arrays

Deletion in an array refers to the operation of removing an element from the array. Like insertion, deletion is also tricky with arrays because arrays in C++ are of a fixed size.

To delete an element from an array, we don't have a direct method. Instead, we'll typically create a new array with a size smaller than the original array by one, and then copy all elements excluding the one to be deleted from the old array to the new one.

Let's look at an example where we delete an element from a specific index in an array:

```cpp
int main() {
    // Original array
    int array[] = {1, 2, 3, 4, 5};
    int deleteIndex = 2; // Index of element to delete
    int originalSize = sizeof(array) / sizeof(array[0]); // Size
        of original array

    // Print original array
    std::cout << "Original Array:" << std::endl;
    for (int i = 0; i < originalSize; ++i) {
        std::cout << array[i] << " ";
    }
    std::cout << std::endl;

    // Create a new array with one less element
    int newArray[originalSize - 1];
    int newSize = 0; // Size of new array

    // Copy elements from old array to new array excluding the
        element to be deleted
    for (int i = 0; i < originalSize; ++i) {
        if (i == deleteIndex) {
            continue; // Skip the element to be deleted
        }
        newArray[newSize++] = array[i]; // Copy elements to new
        array
    }

    // Print new array
    std::cout << "Array after deletion:" << std::endl;
    for (int i = 0; i < newSize; ++i) {
        std::cout << newArray[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

As with insertion, this method is not very efficient, especially for large arrays or multiple deletions. It involves shifting the elements and reallocating memory, which consumes a lot of time and resources. This issue is one of the reasons why we often use more complex data structures like dynamic arrays (such as `std::vector` in C++) for tasks involving frequent insertions and deletions. We will learn about these data structures later in this course.

Merging

Merging Two Arrays

Merging in the context of arrays involves combining two arrays into a single array. Unlike insertion and deletion, merging does not alter the original arrays but creates a new one that holds all elements from both arrays.

The simplest way to merge two arrays in C++ is to first create a new array of the correct size (which should be the sum of the lengths of the two input arrays), then copy over the elements from both arrays.

Here is an example:

```
int main() {
    // Define two arrays
    int array1[] = {1, 2, 3, 4, 5};
    int array2[] = {6, 7, 8, 9, 10};

    // Calculate sizes of the arrays
    int size1 = sizeof(array1) / sizeof(array1[0]);
    int size2 = sizeof(array2) / sizeof(array2[0]);

    // Dynamically allocate memory for the merged array
    int* mergedArray = new int[size1 + size2];

    // Copy elements from array1 to the new array
    for (int i = 0; i < size1; i++)
        mergedArray[i] = array1[i];

    // Copy elements from array2 to the new array
    for (int i = 0; i < size2; i++)
        mergedArray[i + size1] = array2[i];

    // Print merged array
    std::cout << "Merged Array:" << std::endl;
    for (int i = 0; i < size1 + size2; i++) {
        std::cout << mergedArray[i] << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

In this code, we first create a new array of size equal to the sum of sizes of the input arrays. Then we copy all elements from the first array followed by all elements from the second array into the new array. As a result, the new array contains all elements from both input arrays in their original order.

However, this approach does not take into account any sorting or ordering of elements in the input arrays. If your input arrays are sorted and you want the merged array to be sorted as well, you'll need to implement a more complex merging algorithm, such as the one used in merge sort, or sort the merged array afterwards.

We'll explore such algorithms in later parts of this course, where we'll learn about sorting algorithms and their applications.

Formative Assessment 1

Formative Assessment 2